

仲恺农业工程学院

毕 业 设 计

基于单片机和步进电机的智能时钟系统研制

姓 名	吴凯锋
学 院	自动化学院
专业班级	自动化 214 班
学 号	202121724408
指导教师	黄伟锋；胡海滨（校外）
职称/职务	副教授；工程师
答辩日期	2025 年 4 月 26 日

仲恺农业工程学院教务部制

Development of Intelligent Clock System Based on Single Chip Microcomputer and Stepping Motor

Guo Kaifeng

College of Automation

Zhongkai University of Agriculture and Engineering

Supervisor: Associate Prof. Huang Wei Feng

Engineer Hu Hai Bing (Extramural Tutor)

学生承诺书

本人郑重承诺：所呈交的毕业论文（设计）是本人在导师的指导下独立进行研究所取得的成果。除了文中已用特别标志加以标记的引述内容之外，本论文（设计）不含有任何其他个人或集体已经发表或撰写的研究成果。对本文研究做出过重要贡献的个人或集体，均已在文中以明确的方式标明。若在毕业论文（设计）的各项检查、评比中被发现有抄袭、剽窃或其他的违规行为，本人愿按学校有关规定接受处理，并承担相应的法律责任。

学生（签名）：

2025 年 4 月 1 日

摘 要

本设计旨在开发一种基于单片机和步进电机的智能时钟系统,以满足个性化和智能化需求。系统以 STM32F103C8T6 单片机为核心,结合 28BJY-48 步进电机、M38 蓝牙模块和 ESP32-S3-N16R8 语音助手,实现指针与系统时钟同步、闹钟设定及蓝牙连接等功能。硬件设计涵盖电源管理、OLED 显示和音频处理等模块,确保系统稳定运行。软件采用模块化设计,包括时间显示、闹钟设置和语音交互等功能。测试结果表明,RTC 时钟精度高,按键响应灵敏,电源转换稳定。本设计成功构建了一个功能完善、稳定可靠的智能时钟系统,未来可拓展更多功能以提升用户体验。

关键词: 步进电机; 实时时钟; 语音交互; STM32

Abstract

This design aims to develop an intelligent clock system based on single-chip microcomputer and stepper motor to meet the needs of personalization and intelligence. The system is based on STM32F103C8T6 single-chip microcomputer, combined with 28BJY-48 stepper motor, M38 Bluetooth module and ESP32-S3-N16R8 voice assistant to realize functions such as pointer synchronization with system clock, alarm setting and Bluetooth connection. The hardware design covers modules such as power management, OLED display and audio processing to ensure stable operation of the system. The software adopts modular design, including functions such as time display, alarm setting and voice interaction. The test results show that the RTC clock has high accuracy, sensitive button response and stable power conversion. This design successfully builds a functional, stable and reliable intelligent clock system, and more functions can be expanded in the future to enhance user experience.

Keywords: Stepper motor; real-time clock; Voice Interaction; Single chip microcomputer

目 录

1 前言	1
1.1 研究目的与意义	1
1.2 国内外研究情况	1
1.3 研究不足与改进方案	1
1.4 本文设计的创新点	2
1.5 章节安排	2
2 系统硬件设计及选型	4
2.1 单片机选型及原理图设计	4
2.1.1 外部晶振设计	4
2.1.2 复位电路和 BOOT 电路设计	5
2.1.3 单片机供电设计	5
2.1.4 烧录电路设计	6
2.2 步进电机的选型及驱动原理	7
2.3 蓝牙模块选型	10
2.4 语音助手设计及选型	11
2.5 DC-DC 实现 12V 转 5V	11
2.6 系统供电电源设计	13
2.6 LDO 实现 5V 转 3.3V	15
2.7 其余模块的选型思路	16
2.7.1 OLED 模块	16
2.7.2 蜂鸣器模块	16
2.7.3 后备电源设计	16
2.8 系统 PCB 的 Layout 设计	17
3 系统软件设计	19

3.1	系统整体设计思路	19
3.2	OLED 的通信设计思路及 IIC 协议介绍	20
3.3	按键模块的设计思路	21
3.4	实时时钟模块的设计思路	22
3.5	步进电机驱动的设计	24
3.6	语音交互的设计及原理介绍	25
4	试验与数据分析	28
4.1	RTC 时钟精度测试	28
4.2	按键功能测试	29
4.3	电源尖峰电压测试	30
5	结论与展望	32
5.1	结论	32
5.2	展望	32
	参 考 文 献	34
	附 录	36
	致 谢	39

1 前言

1.1 研究目的与意义

随着技术的不断进步，智能产品逐渐融入人们的日常生活，智能时钟作为其中的重要一员，其应用范围和重要性也在不断扩大。智能时钟是传统时钟与现代信息技术结合的产物，通过内置的微控制器和传感器，结合通信技术，实现时间显示、闹钟提醒、语音交互、环境监测等多种功能。它不仅能精确显示时间，还能根据用户需求和环境变化提供个性化服务^[1]。从家庭、办公室到学校、医院等特定场景，智能时钟不仅满足了人们基本的时间显示需求，还通过集成多种功能提升了用户体验^[2]。然而，现有的智能时钟功能虽丰富，但在智能化程度上仍有不足，尤其在语音交互等先进功能的集成和应用方面，尚未充分满足用户对便捷、高效时间管理工具的期待。

目前市场上的智能时钟大多通过不断增加传感器模块来扩展功能，如环境交互和互联网信息获取，但这种简单的功能叠加方式存在诸多问题。一方面，不断增加的传感器模块导致成本上升、集成度降低，且在实际市场应用中尚未广泛普及；另一方面，现有产品的语音交互功能大多仅限于简单的指令识别，缺乏更自然、更智能的交互体验。针对这些问题，本次设计旨在对现有功能进行优化整合，并重点引入语音交互功能，实现更自然、更智能的人机交互，进一步提升用户体验。这种语音交互功能不仅支持基本的指令识别，还能够进行简单的聊天对话，为用户提供了一个更加便捷和有趣的交互体验。

1.2 国内外研究情况

国外在智能时钟领域的研究较为成熟，产品功能丰富多样。例如，一些智能时钟能够与智能手机、智能音箱等设备连接，实现远程控制、语音控制等功能；还有一些智能时钟带有气象预报功能，检测环境质量，能够自动调节亮度等^[3]。此外，国外研究不仅关注智能时钟的功能，还注重用户体验。国内对于智能时钟系统的研究主要集中在高校和科研机构，相关论文和专利数量较多。一些高校开展了基于单片机的智能时钟系统设计研究，实现了时间显示、闹钟提醒、温湿度监测等功能。近年来，国内一些企业也开始涉足智能时钟领域，推出了具有自主知识产权的智能时钟产品^[4]。

1.3 研究不足与改进方案

针对现有智能时钟的不足，本设计对现有功能进行了优化整合，并重点引入语音交互功能。在功能集成方面，采用更先进的微控制器和传感器技术，提高集成度，降低生产成本和功耗，使设备在保持高性能的同时更具市场竞争力^[5]。对于语音交互功能，除了

提升语音识别的准确性和对复杂指令的理解能力外，还特别优化了语音交互模块，使其能够进行简单的聊天对话，为用户提供了一个更加便捷和有趣的交互体验。此外，在用户体验方面，注重用户界面和交互设计的优化，简化操作流程，提高设备的易用性和用户满意度。通过这些改进，本设计旨在提升智能时钟的智能化水平，使其更贴合用户需求，拓展其应用场景，为用户提供一个更加智能、便捷的生活体验。

1.4 本文设计的创新点

本设计开发的智能时钟系统，巧妙融合了传统时钟的机械美感与现代智能技术，实现了多项创新功能。

在计时精度上，系统选用外部低速晶振（LSE）作为 RTC 时钟源，通过智能算法动态校准，确保计时精准，误差控制在消费级应用的合理范围内。

智能语音交互是系统的一大亮点。用户可以通过语音指令设置闹钟、查询天气等，系统借助 ESP32-S3-N16R8 开发板实现语音识别和合成，通过千帆大模型 API 处理指令，提供流畅的语音交互体验。

系统采用 28BJY-48 步进电机结合减速齿轮组，通过 ULN2803 驱动芯片提供大电流驱动，确保电机稳定运行。通过定时器中断智能控制步进电机，实现指针与系统时钟的精准同步。

系统采用模块化设计，将时间显示、闹钟设置、语音交互等功能封装为独立模块，便于功能扩展和维护。结合传统时钟与现代智能功能，提供时间显示、闹钟提醒、蓝牙连接、语音交互等多样化功能。

在电源管理方面，系统优化设计，采用 LM2596S-5.0 DC-DC 转换芯片实现高效降压，将 12V 电源智能转换为 5V，供步进电机使用。输入端接入电解电容和陶瓷电容，吸收尖峰电压，平缓电压输入，同时通过看门狗复位设计防止程序异常，确保系统长期稳定运行。

用户体验方面，系统注重界面和交互设计，通过 OLED 显示屏提供直观的信息展示，通过按键模块实现便捷的功能切换和设置调整。例如，用户可通过按键智能实现功能切换、时间调整等操作。系统在按键读取接口处加入延时接口，排除按键抖动的干扰，确保按键响应的灵敏性和准确性。

1.5 章节安排

作为智能时钟，仍然需要保留传统时钟的功能，并在此基础上进行功能的创新，同时

基础功能仍然可以有进步的空间因此仍然需要对基础功能进行设计以及优化。

第一章为前言，主要介绍本设计的背景、研究目的与意义，并对国内外相关研究情况进行概述，指出现有研究的不足之处和本设计的改进方案。同时介绍本文设计的创新点以及对本报告的章节安排进行简要说明。

第二章为系统硬件设计及选型，详细介绍了本系统硬件部分的设计方案，包括单片机、步进电机、蓝牙模块、语音助手等关键模块的选型及其原理图设计。本章还涵盖了系统供电电源设计、PCB Layout 设计等内容。

第三章为系统软件设计，阐述了系统软件部分的设计方案，包括系统整体设计思路、OLED 通信设计、按键模块设计、实时时钟模块设计、步进电机驱动设计以及语音交互设计等。本章详细介绍了各功能模块的软件实现方法和原理。

第四章为试验与数据分析，展示了系统测试的具体过程和测试结果，包括 RTC 时钟精度测试、按键功能测试、电源转换测试和电源尖峰电压测试等。通过对测试数据的分析，验证了系统设计的正确性和可靠性。

第五章为结论与展望，总结了本设计的研究成果，包括系统实现的主要功能和性能指标，并对未来的改进方向和应用前景进行了展望。

2 系统硬件设计及选型

2.1 单片机选型及原理图设计

设计一个基于步进电机和单片机的智能时钟，首先需要选择合适的单片机，通过查阅资料，最终选择了 STM32F103C8T6 这颗主控芯片，STM32F103C8T6 是 STMicroelectronics 公司基于 ARM Cortex-M3 内核设计的高性能 32 位微控制器，广泛应用于工业控制、消费电子及物联网领域^[6]。

对比同类的芯片，STM32F103C8T6 具有丰富的开发资源和生态成熟度，同时支持 ST-Link、J-Link 等通用编程器，在同类竞品中能以合适的价格提供工业级的外设集成度，并且运算效率为 1.25 DMIPS/MHz，显著优于传统 8/16 位 MCU。

该芯片内部 8MHz 时钟 HSI 最高可倍频到 64MHz，外部 8MHz 时钟 HSE 最高可倍频到 72MHz，包含 37 个 GPIO，每个 GPIO 口最高输出电压为 3.3V，同时板载 2 个 ADC 通道和 4 个定时器，2 个看门狗定时器，并且支持众多常用通信如 USART、SPI、IIC、CAN，并且拥有一个 24 位的滴答定时器。因此颗芯片完全可足够实现该设计。

查阅官方手册得知该芯片的工作电压为 2V~3.6V，常用的 USB 为 5V 电压输出，同时为了实现单片机的时钟源稳定，采用外部晶振作为 STM32F103C8T6 时钟源，该芯片最高倍频倍数为 9 倍，因此外部晶振选型为体积小且输出精度高的 8MHz 无源晶振，并使用 LDO 将电压转换至芯片的工作电压。同时需要包含复位电路以及工作指示灯，最后需要有烧录和调试接口方便开发时进行阶段性测试，因此还需要引出调试接口。

2.1.1 外部晶振设计

振荡器，是一种能量转换装置，可以将直流电流转换为交变电流，交变电流具备一定的频率，有了频率就能够作为一个基准，协调和同步电路中各个部分的工作^[7]。虽然 STM32F103C8T6 内部自带时钟系统，但是其内部时钟容易受到温度和电压等不可控的因素影响，不过该芯片支持使用外部晶振为系统提供时钟。因此为了满足不同场景下的应用，我们采用外接晶振的方法为系统提供工作时钟。常用的晶振频率为 8MHz，同时出于功耗及稳定性的考虑，晶振采用无源晶振。最后为了使晶振运行的更加稳定，需要在晶振两端接入两个小容值电容。原理图设计如图 1

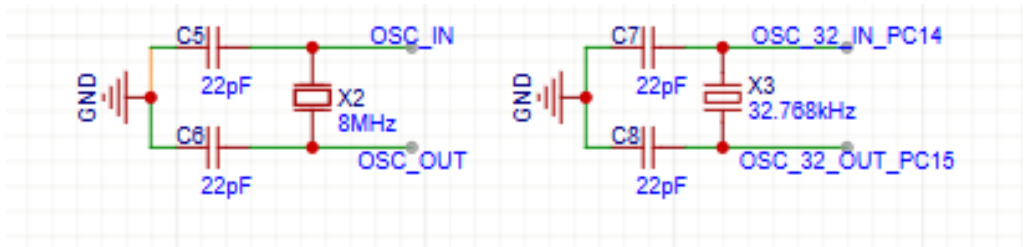


图 1 晶振电路设计

2.1.2 复位电路和 B00T 电路设计

为了防止程序运行时出现卡死或跑飞的情况,应设置复位按键方便芯片进行复位重启。并且为了保证在复位后能正确的加载引导程序,保证芯片各硬件的初始化正确,还需要设计 B00T 电路,参考 STM32F103C8T6 的数据手册得知,对于正常的使用芯片,只需要将 B00T0 和 B00T1 短接即可,此时当系统复位时,将会从 FLASH 加载和执行程序。设计如图 2 所示

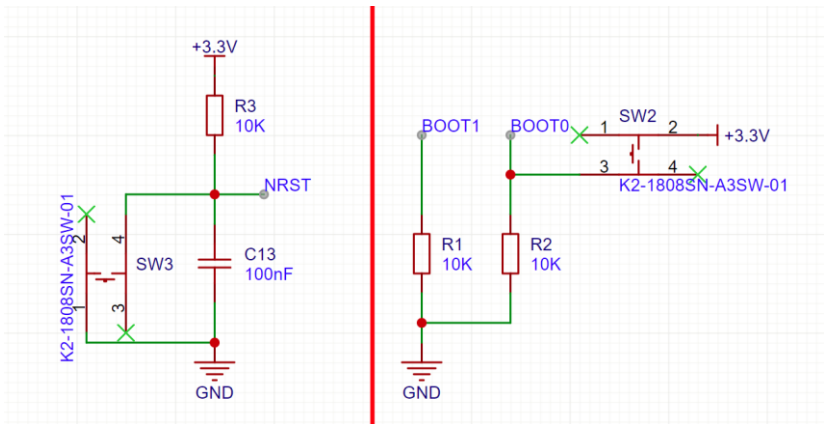


图 2 复位电路（左）及 B00T 电路（右）

2.1.3 单片机供电设计

STM32F103C8T6的工作电压为3.3V,而通常与之通信的外设接入的电源一般为5V,因此,为了保证能与外设兼容,需要设计 5V 转 3.3V 电路,常用的转换芯片类型为 LDO,因此选用 AMS1117 降压芯片进行降压,AMS1117 的输入电压范围在 4.5V 至 12V 内,因此主流电源的纹波并不会直接影响单片机的工作。电路设计图 3 所示。

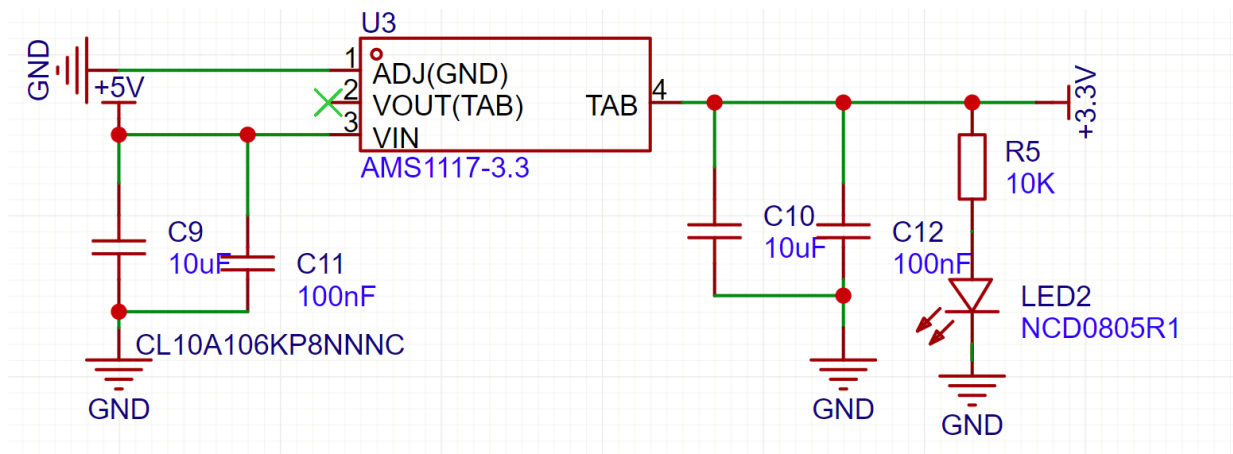


图 3 AMS1117 降压电路设计

同时为了方便给芯片供电，便于使用。还需要设计供电口，供电口一般与 LDO 直接进行连接。电路设计如图 4 所示。

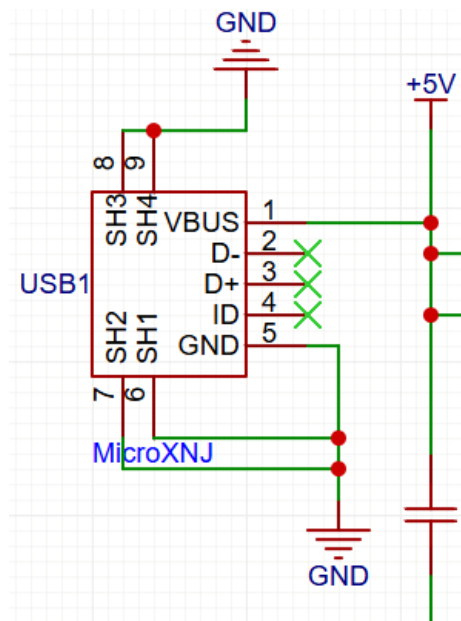


图 4 USB 供电电路设计

2.1.4 烧录电路设计

为了方便将编写好的程序烧录至芯片中，STM32F103C8T6 支持使用 SWD 调试标准及 JTAG 调试标准。相较于 JTAG，SWD 只需要占用 SWDIO 及 SWCLK 两根通信线，而 JTAG 需要占用四根或五根通信线。并且在简单使用时并不需要使用 JTAG 的复杂调试功能，SWD 串行调试由一个双向数据线和单向时钟线组成实现数据的同步传输，可替代

JTAG 调试。时钟频率为 50 MHz 时数据传输速率可达 4 MB/S，较少的 PCB 布局布线资源便可实现低功耗低成本应用^[8]。因此出于对资源节省以及电路设计的原因，采用 SWD 调试标准，同时需要将该引脚引出方便调试。电路设计如图 5 所示。最终实物图如图 6 所示。

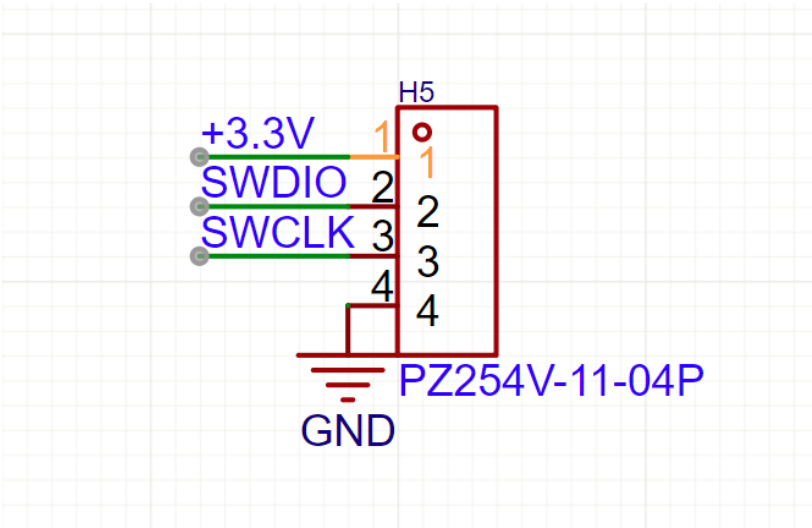


图 5 SWD 下载口电路设计

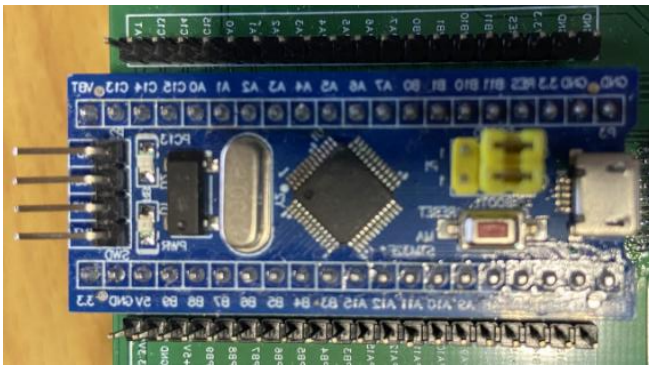


图 6 STM32F103C8T6 最小系统板实物图

2.2 步进电机的选型及驱动原理

步进电机的基本结构较为复杂和多样，且组成部分之间有着紧密的联系。步进电机的基本结构包含许多部件，包括但不限于前端盖、转子铁芯、磁钢、绕组绝缘、定子铁芯、波纹垫圈、轴承、中心轴、后端盖、螺钉。其中绕成圈状的金属丝称为螺线管，定子齿槽上的金属丝可以称之为相，也可以称之为线圈^[9]。

常用的步进电机有四相和两相，出于体积和实现方式，最终选择 28BJY-48 步进电机，

该电机是一款五线四相的的步进电机，内部包含定子及转子以及一组 64:1 的减速齿轮组，每个定子包含 8 个齿，每对对称齿上的线圈串联形成 4 相绕组，通过不同相序通电驱动转子转动，相序的通电方式可以由单片机进行控制。图 7 为该电机的内部结构示意图。从此图可以得知每相都与公共端相连，当某相导通，也就相当于开关闭合时，该相导通并且线圈得电产生磁性吸引转子转动。由此可得当每相交替导通时，即可实现步进电机的旋转。控制每相的逻辑可以交由单片机处理。

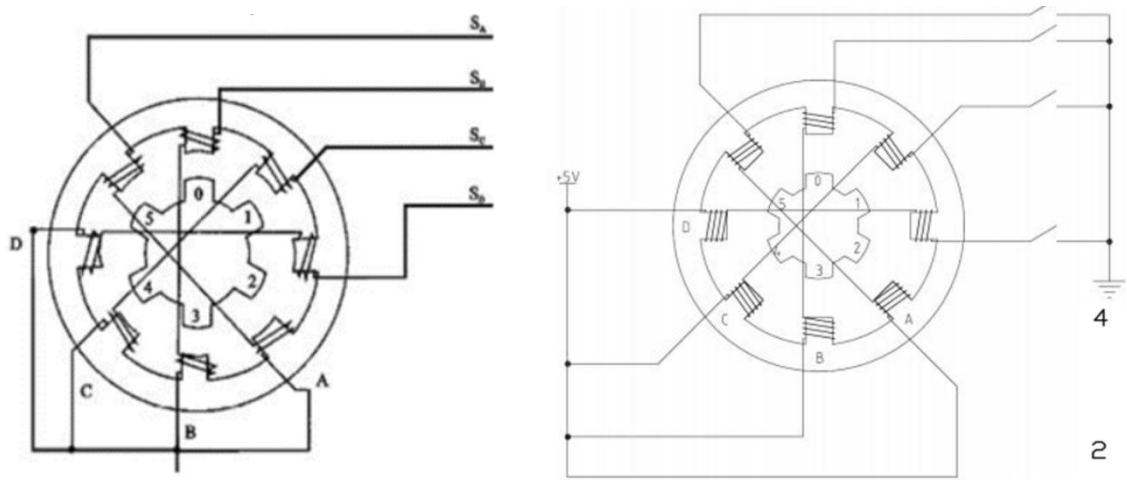
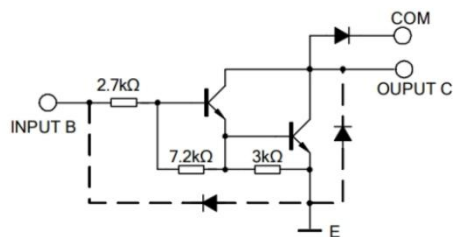
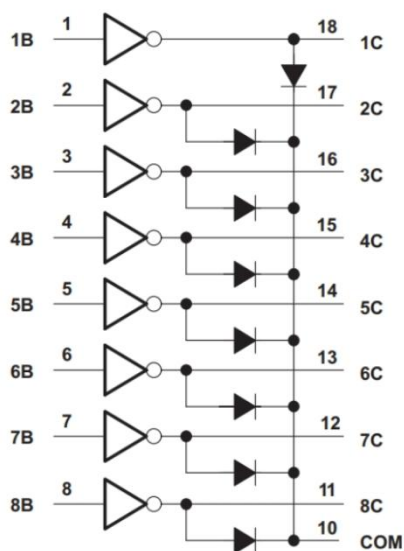


图 7 28BJY-48 步进电机内部结构示意图

由于驱动电机所需要的电流较大，对于 28BJY-48 步进电机，每相所需要的电流都比单片机的输出电流大。因此单片机的 GPIO 引脚不能直接连接步进电机，而是需要连接驱动板输出大电流进行驱动。常用的驱动芯片有 ULN2003，但是该芯片只有 7 个通道，并不足以同时支撑两个步进电机的驱动^[10]。

因此我们选择具有 8 通道的 ULN2803 来驱动两个步进电机。ULN2803 是专为低压系统设计的大电流达林顿管阵列，电路由八个独立的达林顿管组成，达林顿管通常由两个 NPN 或 PNP 晶体管组成，它们的基极和集电极相连，而发射极则分别连接到外部电路。这种连接方式使得第一个晶体管的集电极电流成为第二个晶体管的基极电流，从而实现电流的二次放大。每个达林顿管带有续流二极管，可用于驱动继电器、步进电机等感性负载。单个达林顿管在输入电压低至 1.8V 状态下支持电流 500mA 输出，将达林顿管并联可以得到更大输出电流能力。该电路广泛应用于继电器驱动、照明驱动、显示屏驱动、步进电机驱动和逻辑缓冲器。图 8 为 ULN2803 的芯片内部电路逻辑图。



Note: The input and output parasitic diodes cannot be used as clamp diodes.

图 8 ULN2803 逻辑图（左）及局部示意图（右）

从图 8 可以得知，应在 INPUT B 端接入 GPIO，当 GPIO 输出高电平时晶体管 T1 导通，此时 T1 的发射极电流经过放大后流入晶体管 T2 的基极，此时 T2 导通，引导集电极电流流向地，此时输出端相当于接地因此表现为低电平。同时电流经过两个晶体管放大后足以驱动步进电机转动，因此 ULN2803 具有很强的灌电流能力。同理当 GPIO 输出低电平时，两个晶体管均不导通，故此时芯片与步进电机没有形成回路，输出端表现为高阻态因此步进电机内部线圈不得电不会吸引转子转动。

因为步进电机内部线圈可以看成一个电感，因此在线圈失电时会产生反电动势，通过图 3 可以发现在公共端和输出端接入了一个续流二极管，当反电动势产生时，可以通过该回路消除反向电流保护芯片，同时为了使续流电流尽量小，因此公共端需要接高电平，一般接步进电机的公共端电源^[11]。

驱动原理如图 9 所示。在 ULN2803 的输出端外接 LED 指示灯表明步进电机的运动状态，同时加入限流电阻防止 LED 烧毁^[12]。实物图如图 10 所示。

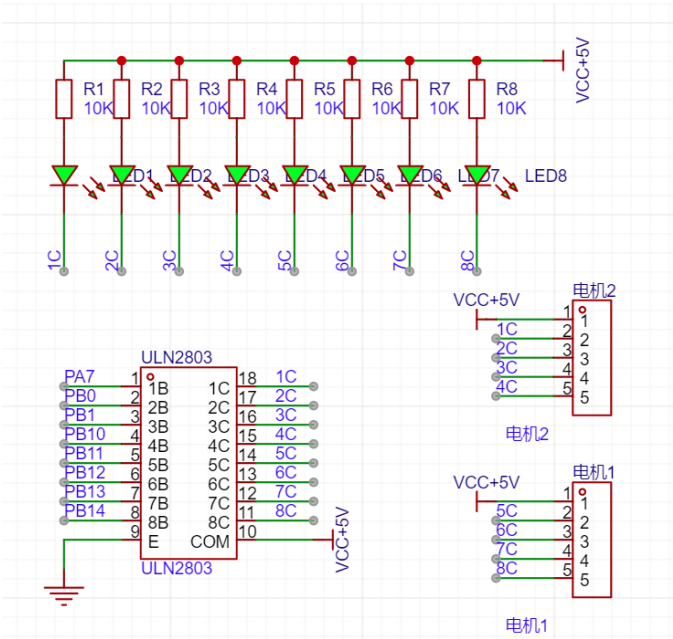


图 9 双电机驱动电路设计

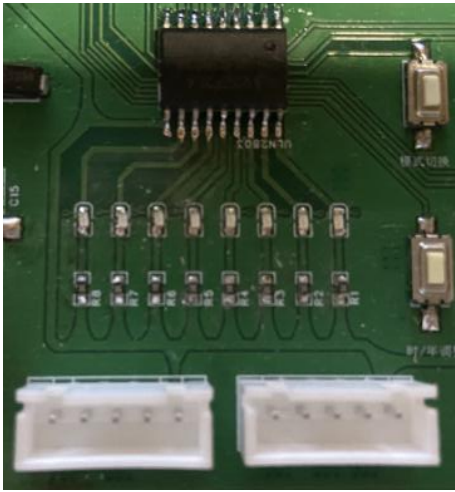


图 10 双电机驱动电路实物图

2.3 蓝牙模块选型

蓝牙选型首先需要确定蓝牙协议，因为需要能播放手机的音频，因此首先该模块需要有解码音频数据的能力，同时集成功放芯片。因此选择 M38 模块使用，该模块使用杰理旗下的 AC24BP24867-65A4，该芯片支持多种音频格式，如 MP3、WMA 等，支持蓝牙 5.1 协议,该协议具有高稳定性、低功耗性等特点，并且向下兼容已存在的蓝牙协议^[13]。M38 模块的功放部分采用了 XS9971 芯片，这是一款低电磁干扰（EMI）的音频功率放大

器，无需外接滤波器。在 5.5V 工作电压下，XS9971 能够向负载提供最大 5.5W 的功率，此外，该芯片还具有过热自动关断保护机制，能够在温度过高时自动关闭，保护芯片不受损坏。其应用电路简单，输出端不需要外接耦合电容或上举电容和缓冲网络，这使得其在使用上更加方便。

2.4 语音助手设计及选型

语音助手的实现主要依靠与模型的处理能力以及硬件的容量和数据处理能力，在前文提及的 STM32F103C8T6 芯片内部 FLASH 只有 64KB，同时并不具备天线，因此需要搭载天线模块接收和发送数据，其次语音助手的使用势必需要使用常用的 I2S 协议，而在 STM32F103C8T6 的数据手册中并未明确提及支持 I2S 协议，同时还需要兼顾与 OLED 和步进电机以及蜂鸣器的逻辑交互，处理压力相对较大，因此考虑使用更高性能的处理芯片，最终采用国产的 ESP32-S3-N16R8 开发板，相比于 STM32F103C8T6，ESP32-S3-N16R8 板载了天线，支持 802.11 b/g/n 和蓝牙 5.0，并且支持丰富的协议如 SPI、I2C、UART、I2S 等，同时 FLASH 容量高达 16MB，主频高达 240MHz，具有 8MB PSRAM 和向量指令加速神经网络运算，适用于语音唤醒、人脸识别等 AI 应用。同时 ESP32 拥有成熟的开发生态和众多的参考功能实例，便于进行快速开发和功能验证。

语音交互还需要有输入输出设备，因此还需要选取麦克风以及喇叭，一些开发板会自带这些外设，从而达到压缩产品空间的目的^[14]。但为了保证音频的输出质量，需采用独立的外设和音频功放对输出的音频信号进行处理，最终确定采用 MAX98357 音频功放模块以及 INMP441 麦克风模块。

INMP441 中包含一个微型膜片，以及 ADC（数模转换器），并且支持硬件 I2S。当声波作用于膜片时，膜片发生振动引发电容变化。电容变化被转换为电信号，经过内部的模拟前端（AFE）放大和处理后，输出为数字音频信号。

MAX98357 是一款高品质、低功耗的单声道 D 类音频放大器芯片，具有内置的数字音频接口，能够将数字音频信号解码并放大后直接驱动扬声器。

2.5 DC-DC 实现 12V 转 5V

整个时钟系统电路有多种工作电压，如 OLED 模块使用的 3.3V 与步进电机使用的 5V，又因为步进电机的驱动需要较大的电流保证，因此采用 12V 锂电池作为输入电源

对系统进行供电，并在系统中对输入电源进行降压处理。首先需要将电源使用 DCDC 芯片转换为 5V 供步进电机使用，采用 DCDC 芯片的好处是转换效率高，并且可以调节反馈电阻的阻值改变输出电压，同时具有输出电流大，发热少等优点。

翻阅各类 DCDC 芯片的数据手册后决定采用 LM2596S-5.0 这颗芯片。该芯片在该芯片手册中有典型的 12V 转 5V 参考电路，同时芯片支持 4.5V~40V 的宽电压输入，3A 的大电流输出，可以保证电机的稳定运行并且稳定输出 5V，电路设计如图 5。

输入端的输入电容使用一颗 330 μ F 的电解电容以及一颗 0.1 μ F 的陶瓷电容,是因为该模块的 12V 输入电源直接连接至开关,当拨动开关至导通的瞬间会产生尖峰电压,数值大概在 15V 左右。尖峰电压会影响元件寿命甚至损坏元件。而电解电容可以有效的吸收尖峰电压，平缓电压输入。相比于普通的陶瓷电容,电解电容具有更大电容容量。同时电解电容的耐压范围较宽，可以承受更宽的电压波形且能够在长时间内可靠的工作。

在芯片的输出端首先接一颗 SS56A 的肖特基二极管。因为 LM2596S-5.0 是一颗 DCDC 开关芯片，因此当芯片运行了一段时间后，当芯片内部的场效应管处于关断状态时，积累在后续电路中的电感会因电流的减少而产生感应电势并为负载供电，此时因为芯片内部的开关处于关断状态，因此电感与负载间无法形成回路，在电感两侧会有很高的感应电势，很容易就将 LM2596S-5.0 损坏，因此需要使电感两端导通，并且在 LM2596S-5.0 工作时并不会通过该回路连接至负载。因此可以利用二极管的单向导电性，在电感两端接入一个肖特基二极管。对比于普通的二极管，肖特基二极管有更低的正向导通电压以及更快的恢复时间。这意味着可以带来更高的转换效率。在降压电路的最后接入一颗 330 μ F 的电解电容使输出电压更加的平滑和稳定。电路设计如图 11，实物图如图 12。

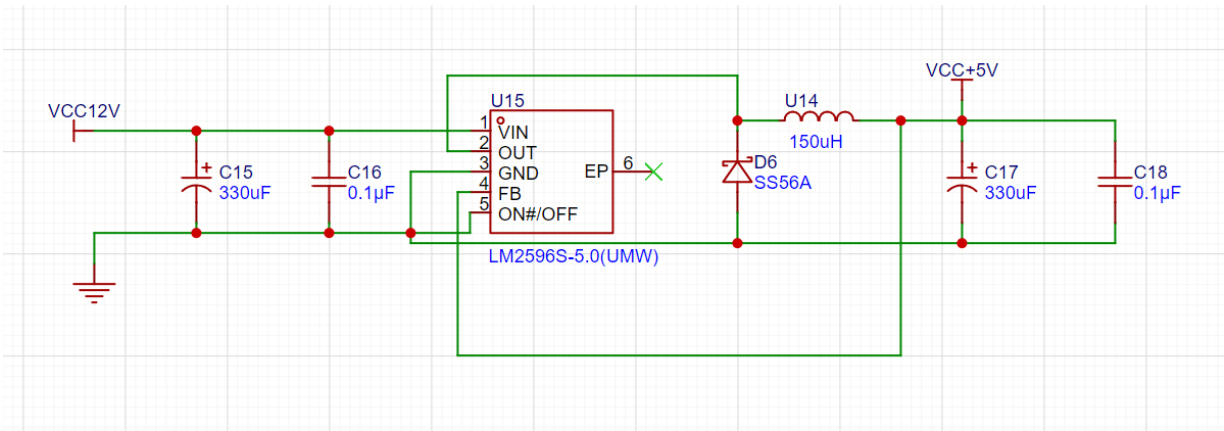


图 11 12V 转 5V 电路设计



图 12 12V 转 5V 电路实物图

PCB 布局方面，LM2596S-5.0 的输出端应该与 SS56A 和电感相近,同时电感下不宜走线防止干扰,应在电感区域采取禁止铺铜的方式消除干扰。在芯片的输入端，因为输入为 12V 电源。应采取局部铺铜的方式接入电源以提高抗干扰能力及增强散热性能^[15]。PCB 布局设计如图 13 所示。

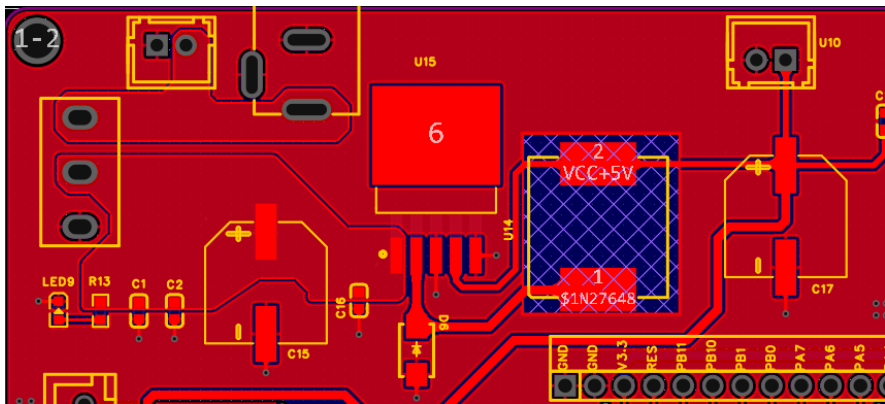


图 13 12V 转 5 电路的 PCB Layout 设计

2.6 系统供电电源设计

为了保障供电更加的便捷，减少操作电池的次数。需要在供电处放置开关，并且为了拓宽供电场景，应提供多种的供电接口方便供电，最后当开关导通后，还需提供标识表明供电正常，并且在DCDC 芯片进行转换前应先将输入电源进行滤波后再接入降压电路，因此供电设计如图 14 所示，提供了 DC 端接入以及针对板的接入方式进行供电，并采用

LED 表明供电状况。

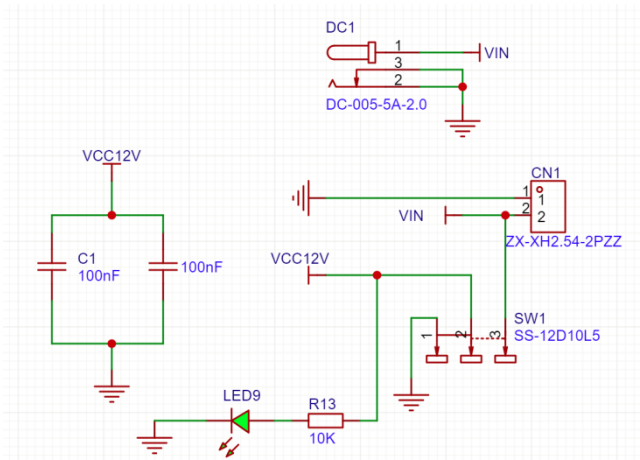


图 14 12V 供电口电路设计

电源部分应采用局部铺铜的方式，以提升散热性并承受大电流的流过。同时电源接入应在 PCB 边缘便于接入电源^[16]。电源指示灯放置于开关下便于观察。PCB 布局设计如图 15 所示。

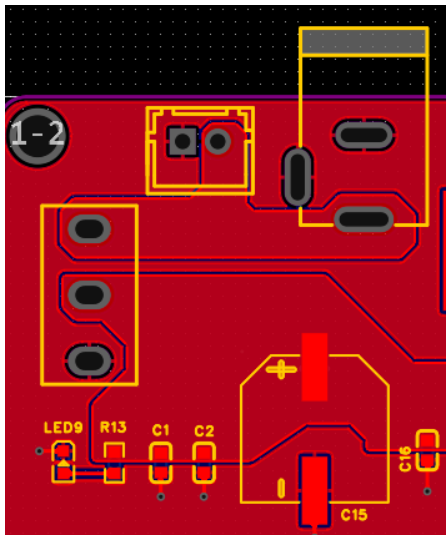


图 15 供电电路 PCB Layout 设计

最后为保证系统的后续拓展，应将各阶段的电源电压进行引出，为外界提供多种电源范围。电路设计如图 16 所示。

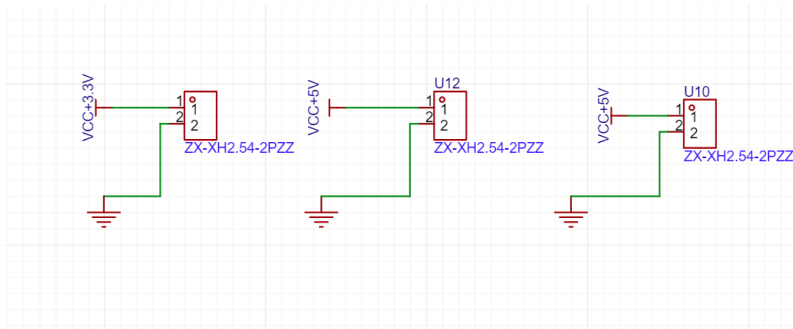


图 16 预留电源电路设计

2.6 LDO 实现 5V 转 3.3V

因为 STM32F103C8T6 的工作电压为 3.3V，在不直接使用 USB 供电的情况下，无法直接使用 5V 电源。因此还需要将转换后的 5V 电压转换 3.3V 才可使用。因为 5V 转换到 3.3V 的降压范围较小，同时为了压缩 PCB 空间，因此使用 LM1117IMPX-3.3 这颗 LDO 进行电源的转换，LM1117IMPX-3.3 的最大输入电压为 20V，同时线性调整率最大为 0.2%。相对于 DCDC，LDO 采用简单粗暴的方法，通过调整内部三极管的导通程度将输出电压稳定在一个波动很小的目标值附近。同时 LDO 能够有效抑制输入电源中的纹波和噪声，确保输出电压的稳定性。并且对外围元件要求较少。LM1117IMPX-3.3 是一颗不可调的 LDO，即分压电阻集成在芯片内部中。因此在我们上一级的输出电压中，并不需要绝对稳定在 5V，LM1117IMPX-3.3 可以动态的调节内部三极管的导通程度使得输出为 3.3V。设计电路如图 17 所示，实物图如图 18。

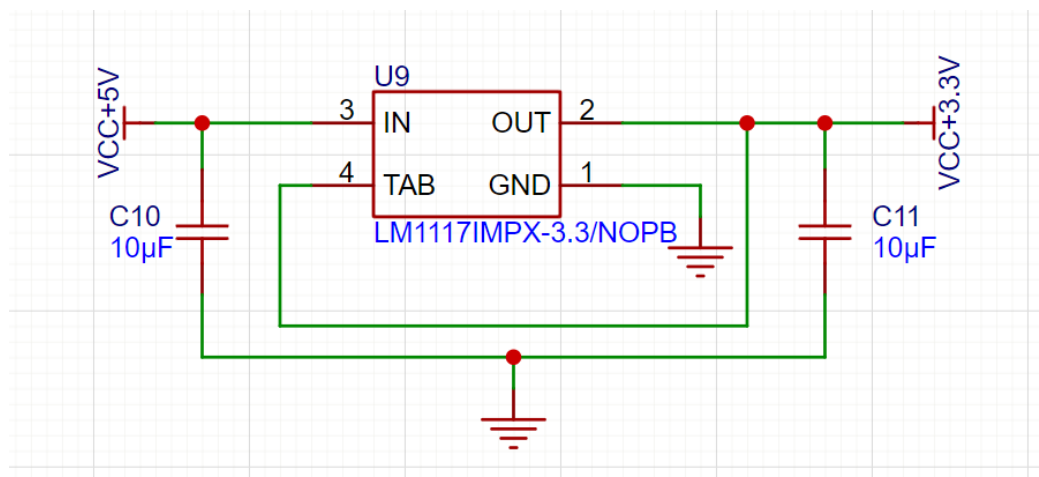


图 17 5V 转 3.3V 电路设计



图 18 5V 转 3.3V 电路实物图

2.7 其余模块的选型思路

2.7.1 OLED 模块

智能时钟系统在保留了传统指针的同时也可以突出数字感，因此选取 OLED 用于显示实时时间以及智能实时时钟的功能，在工程中常用 IIC 协议与 OLED 通信，IIC 协议中规定 SCL 和 SDA 线必须通过上拉电阻连接至 VCC，确保总线空闲时为高电平。因此可以选取 0.96 寸且自带上拉电阻的 OLED 模块。

2.7.2 蜂鸣器模块

因为只需要实现警报功能，因此可以选择有源蜂鸣器，相比于无源蜂鸣器，有源蜂鸣器简单易用，适合固定音调场景。同时需要注意蜂鸣器选择低电平触发的模块，并且 VCC 引脚连接 3.3V，连接 5V 会导致蜂鸣器模块上的三极管始终大于 0.7V 不受单片机的 GPIO 控制而自主发声。

2.7.3 后备电源设计

因为 STM32F103C8T6 本身并不具备后备电源，局部电源树如图 19 所示，当系统得电时，VBAT 可以通过 VDD 获取电源，此时不从 VBAT 引脚获取电源。而在系统掉电后，若没有后备电源，VBAT 引脚不得电，滴答定时器并不工作。因此在 VBAT 引脚还需要外接电池进行供电以保证时钟在断电后仍能继续计时^[17]。VBAT 引脚的工作电压为 1.8V~3.6V，同时工作电流小，因此为了保证时钟的实时性，可采用纽扣电池为 VBAT 引脚供电。

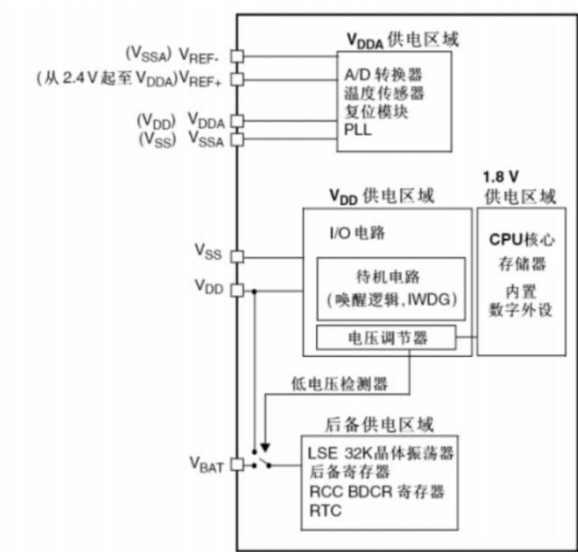


图 19 STM32F103C8T6 后备电源局部电源树

2.8 系统 PCB 的 Layout 设计

PCB 的布局是决定系统能否正常工作中至关重要的一环，好的布局可以有效提升信号的传输并且降低功耗。同时在适宜的位置进行铺铜可以有效的减少电磁干扰^[18]。

采用分模块化进行布局，电源的输入和转换部分布局在 PCB 上方。步进电机引出口为方便接线布局在 PCB 的左侧。

12V 转 5V 部分电感底部尽量不要走线以减小干扰同时可将 LDO 的转换部分一并放置在 5V 输出口后形成完整的转换部分。

为方便交互，0.96 寸的 OLED 放置在按键模块的上方，并利用排母引出提升 OLED 的高度便于获取信息。在 OLED 下方可放置为 VBAT 引脚供电的纽扣电池。

对于 STM32F103C8T6 可采用排母将其固定，当出现问题时便于更换。并将各个引脚通过排针引出，可用于后续的功能拓展。

步进电机部分将其信号线同 OLED 指示灯放置在一起便于实时显示工作状态以及及时发现故障问题。

最后可在 PCB 四周加上过孔便于后续 PCB 的固定以及组装。整体布局如图 20 所示。

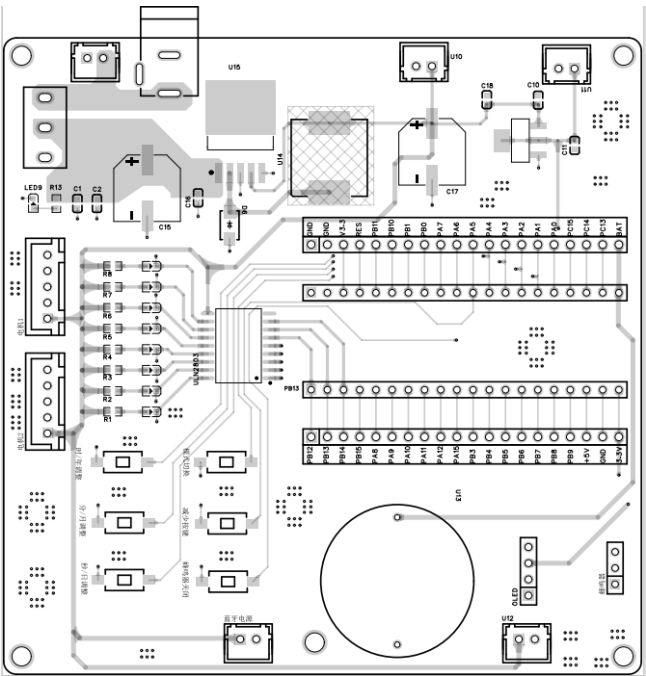


图 20 本文智能时钟系统整体 PCB Layout 设计

3 系统软件设计

3.1 系统整体设计思路

智能时钟首先需要屏幕的显示，因此主控需要同 OLED 进行通信，并且在 OLED 上显示时间以及功能。与 OLED 常用的通信协议为 IIC 协议，该协议是一种串行数据总线。具有硬件资源节约，协议设计精巧，易用，多用与传输命令等优点。整个协议中通信设备之间只需要连接两条线,并且可以挂载多个设备进行通讯^[19]。因此适合与 OLED 此类设备进行通信。

每个步进电机需要 4 个 GPIO 进行驱动，并且控制逻辑简单，只需要定时输出高低电平。因此应尽量选择没有复用功能的 GPIO 用于控制步进电机，按键和蜂鸣器的控制逻辑也较为简单，因此也应尽量选择没有复用功能的 GPIO 用于控制。

每个按钮执行掌管相应的功能，并且都需要将当前的功能显示在 OLED 中，同时在系统重新上电后，需要将 RTC 寄存器中的时间实时显示在 OLED 上。主循环中需要循环的获取每个按键的状态。流程图如图 21 所示。

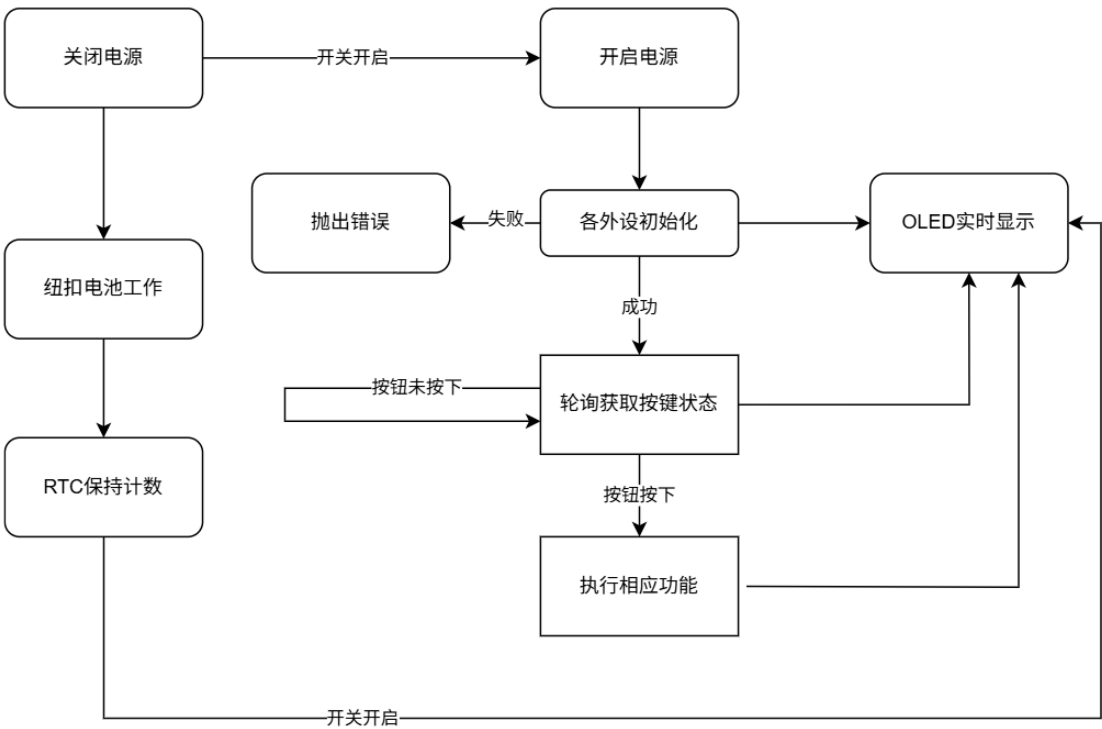


图 21 智能时钟系统流程图

3.2 OLED 的通信设计思路及 IIC 协议介绍

OLED 可以正确显示字符的关键是拥有该字符的字库，OLED 实际上是由一个一个的像素点组合而成。字库就是用以表示每一个字符在单位矩阵中应该如何显示的库。

IIC 协议规定连接在线路中的每一台设备都需要设置为开漏输出(OD)，同时通信线应当接入上拉电阻保持通信线维持在高电平，在与 OLED 通信的应用场景中，常采用一主一从的方式进行通信，即 OLED 作为从设备接收来自主设备的数据，并返回响应位。当通信静默时不能向主设备发送数据。IIC 的通信原理是当主设备需要发送数据时，在时钟线 SCL 为高电平时拉低数据线 SDA 作为起始条件，表明将要在数据总线上传输数据。随后开始发送数据，每发送 8 位数据后从设备需要发送一个应答信号或非应答信号表示是否需要继续接收数据。若不需要接收数据，主设备应在发送完数据后的下一个时钟信号为高电平时拉高数据线 SDA 作为结束标志^[20]。至此一主一从模式下的一次通信完成。

OLED 模块应选择自带上拉电阻的模块便于开发，目前许多 OLED 模块都会引出 SDA 及 SCL 通信线，因此支持硬件 IIC，同时 OLED 的工作电压在 3.3V~5V，因此可以直接同主设备 STM32F103C8T6 中支持 IIC 协议的引脚连接，但出于便于 PCB 布线的原则，也可以采用软件模拟 IIC 协议的方式进行通信。

实现通信的过程首先使用宏定义将通信的 GPIO 进行封装，便于代码的可移植性和可读性。将 IIC 协议的配置、OLED 的初始化、字符、字符串、数字等多种信息的显示功能分别封装成独立的函数，便于调用和维护。初始化模块中将 GPIO 的输出模式设置为开漏模式，字符显示模块中通过设置光标位置和调用字体表，将字符的点阵数据逐行写入 OLED^[21]。以上接口均遵守 IIC 协议，在 IIC 协议的配置模块中，提供了 I2C 通信的基本操作，包括启动信号、停止信号和字节发送接口。

程序流程图如图 22。

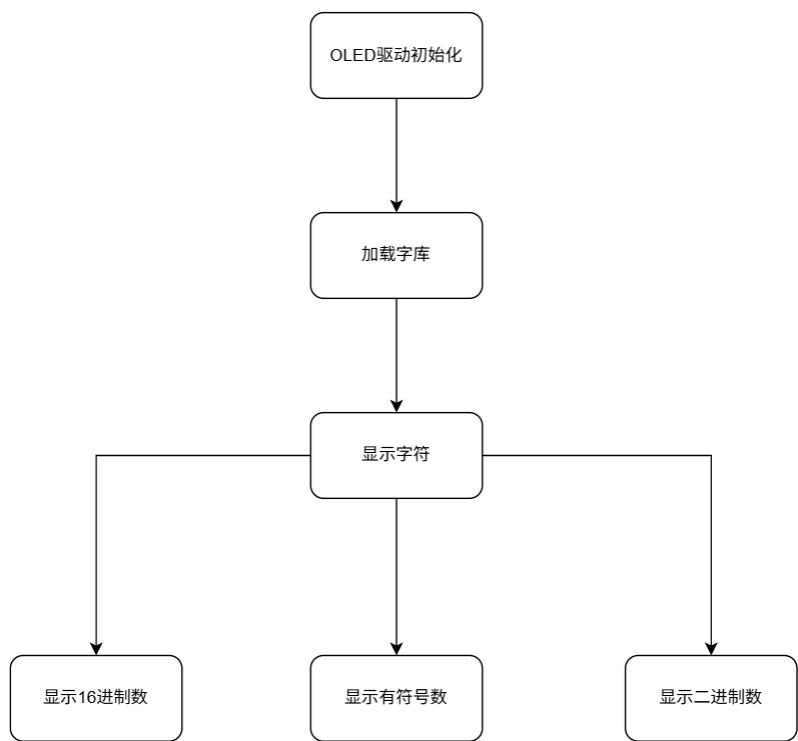


图 22 OLED 设计流程图

3.3 按键模块的设计思路

按键控制作为整个系统中必不可少的一部分，其中的接口应当重复调用以响应外界的任何异步操作。并且还实现时钟的日期调整以及模式切换，并且需要将调整后的效果同步至 OLED 及步进电机。功能切换后，按键的功能也应该同步切换

使用宏定义将按键对应处理的事件进行封装，便于处理各功能需要执行的事件，采取六个按键用于和系统进行交互，实现的功能分别为功能切换，秒/日调整，分/月调整，时/年调整，数值减少，复位蜂鸣器。因为数量较少故可不采用矩阵的形式相连，每一个按键一端均与单独的 GPIO 口进行连接，另一端与参考地相连，当按键按下后，GPIO 直接与参考地连接，端口电平被下拉至低电。因此对于所使用的 GPIO 口配置时应设置为上拉输入，并在主线程中不断读取每个按键的端口电平，当按键按下后可处理相应的环节。

同时考虑到实际按键中会出现抖动，因此还需要在按键读取接口处加入延时接口以排除干扰。

流程图如图 23 所示。

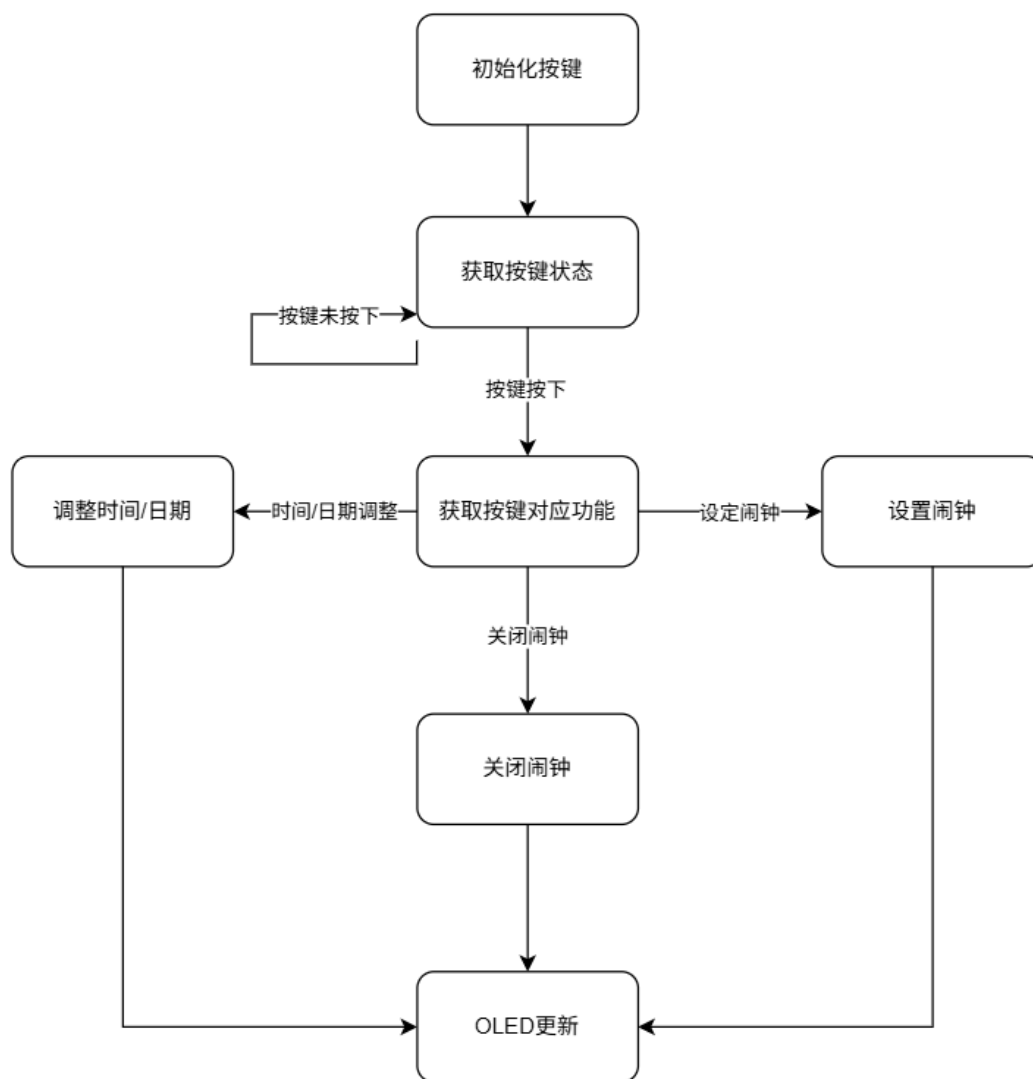


图 23 按键功能流程图

3.4 实时时钟模块的设计思路

实时时钟(RTC)也称为时钟芯片，是一种广泛应用的电子器件，用于提供精确的实时时间，或者为电子系统提供时间基准。大多数时钟芯片，均采用精度较高的晶体振荡器作为时钟源，因此，晶振的频率误差大小，决定了计时系统的性能^[22]。

实时时钟可以通过配置 STM32F103C8T6 内部的滴答定时器实现，同时实时时钟从始至终应当只初始化一次，因为每一次初始化时都会将原本设定的值写入 RTC 计数器，就会导致时间的偏移。因此可以使用备份寄存器进行过滤，在 STM32F103C8T6 中，若 VBAT 在系统掉电后仍然能提供电源，则可以为备份寄存器、RTC、LSE 振荡器（外部低速振荡器）和 PC13 至 PC15 供电。保证主电源被切断后 RTC 可以正常工作。因此在系统第一次初始化时可以往（BKP）备份寄存器中写入特定值用于置标志位，这样一来系统每

一次被主电源接管时，在 RTC 的初始化中可以判断 BKP 是否有值且是否等于掉电前写入的值从而决定是否重新初始化。电源示意图如图 24。

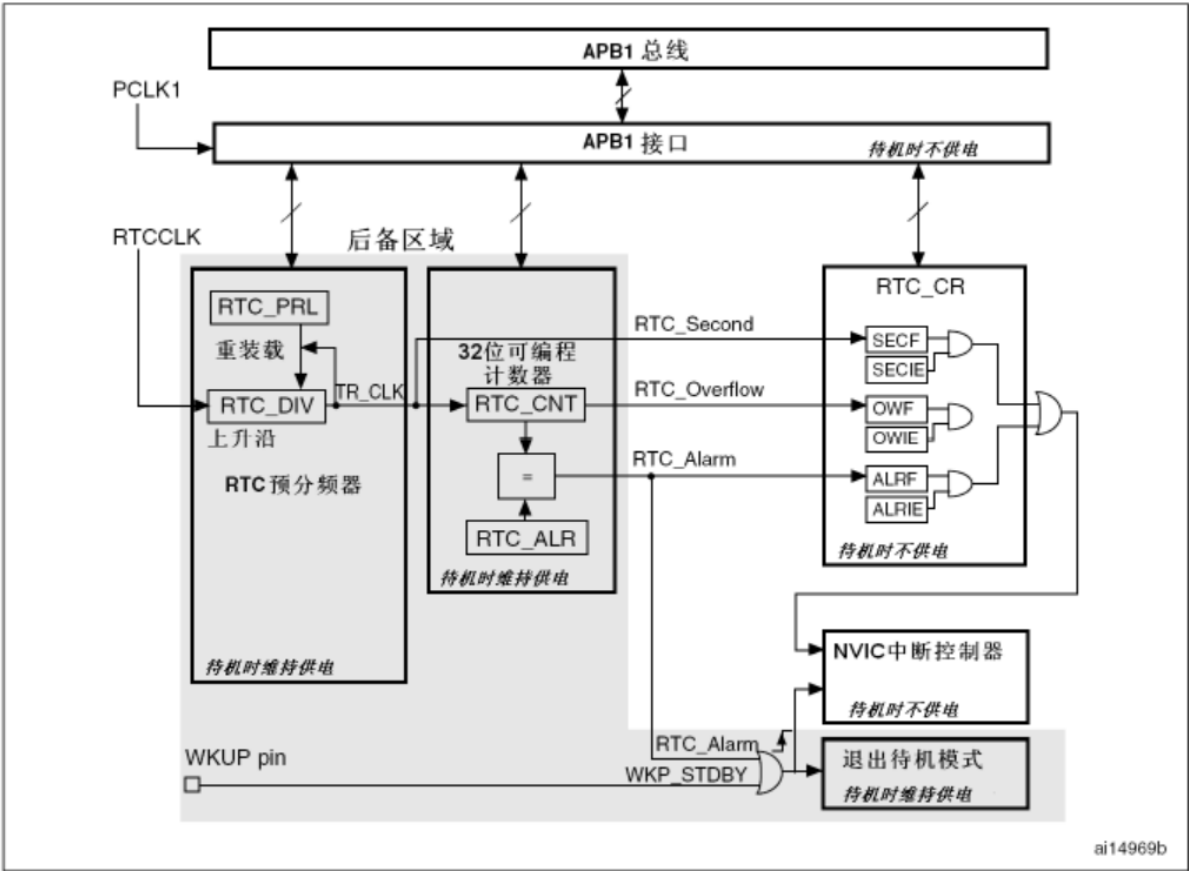


图 24 RTC 工作原理图

RTC 计数器是一个 32 位的寄存器,在 RTC 计数器中的数是以时间戳的形式保存的,这种保存方式对于芯片来说是友好的,但是对于人类来说并不友好,因此需要将时间戳转换为人类易于理解的形式,同时在设置时间的时候,对 RTC 计数器写入值的类型也应该为时间戳,因此需要一个结构体变量充当转换的桥梁。在需要设置时间时,只需要写入日期,而在类型转换的接口中将结构体转换为时间戳并写入 RTC 计数器。同理该接口也需要实现时间戳到日期的转换逻辑。

RTC 时钟源使用 LSE 外部低速时钟,使用 LSE 的原因是因为 LSE 的高精度,LSE 通常采用 32.768 kHz 的石英晶体振荡器,其精度可以达到 ± 20 ppm (百万分之一)甚至更高,相比之下,内部低速 RC 振荡器 (LSI) 的精度较低,大约为 $\pm 1\%$ 。

并且 LSE 在功耗及稳定性都具有出色的表现,LSE 的工作频率较低 (32.768 kHz),

这对于需要长时间运行且对功耗敏感的应用非常有利。虽然 LSI 也是低速时钟源，但其功耗相对较高，且精度不如 LSE。

因为 LSE 是外部晶振，因此 LSE 受温度和电源电压波动的影响较小。这使得 LSE 在不同的工作条件下都能保持较高的计时精度，相比较于 LSI，其是内部 RC 振荡器，其频率会随温度和电源电压的变化而漂移，稳定性较差。

同时在时钟进行模式切换时，若按键连接则应当实现非阻塞轮询，否则会干扰 RTC 的准确性^[23]。

流程图如图 25。

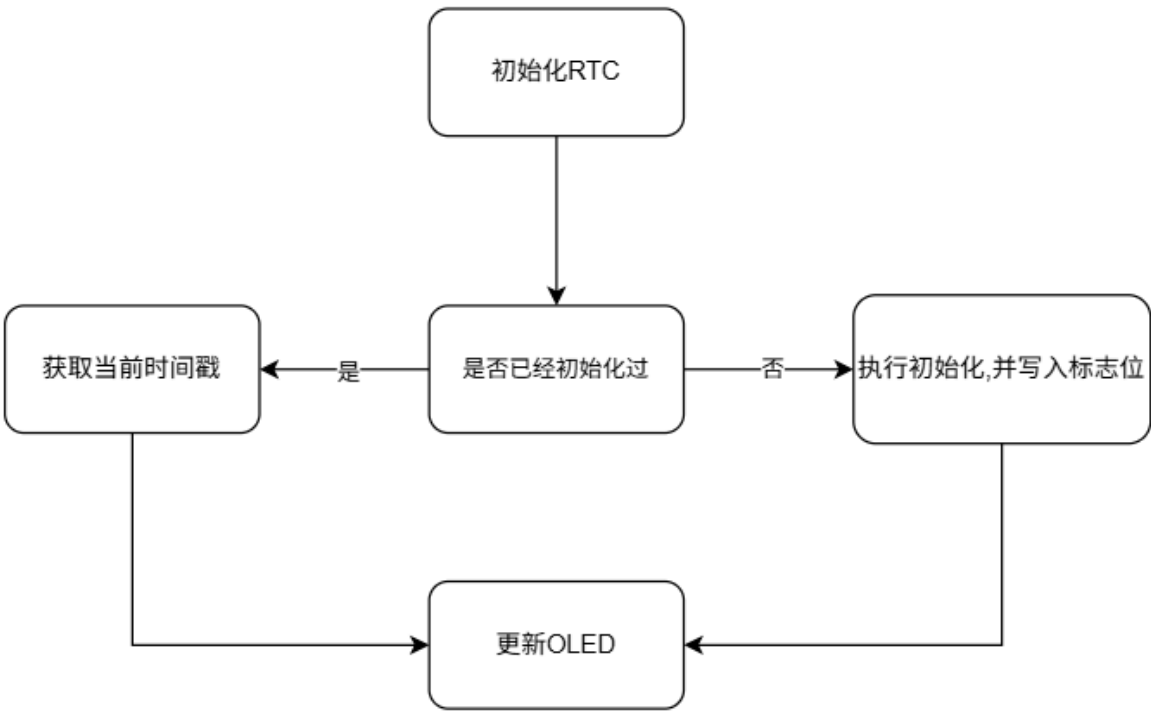


图 25 实时时钟功能设计流程图

3.5 步进电机驱动的设计

步进电机的转动因其需要跟随实时时钟，故接口的驱动逻辑应该尽可能简单并且耗时较少以防止阻塞主线程的运行，同时因为步进电机的过冲失步特性，因此驱动接口中步进电机每一步执行后需要加入适当的延时保证步进电机的平稳旋转。

因为步进电机转动的角度与时钟之间可以存在映射关系，并且对于步进电机驱动的调用频繁，因此可以考虑使用定时器溢出中断调用驱动函数，对于五线四相步进电机的驱动方式有四相八拍，四相四拍，其中四相四拍又可细分为一拍中给单个线圈通电还是

给两个线圈供电。对于 28byj-48 步进电机，其基本步距角为 5.625° ，但由于减速齿轮的存在，假定实际步距角为 Angle,则每一步转过的角度有：

$$\text{Angle} = \frac{5.625}{64} \approx 0.08789^{\circ} \tag{式 1}$$

由于每步转角为 0.08789° ，假定转动一圈的脉冲数为 Pulse，则有：

$$\text{Pulse} = \frac{360^{\circ}}{0.08789^{\circ}} \approx 4096(\text{步}) \tag{式 2}$$

因此想要控制步进电机跟随指针，只需要在 60 秒内走完 4096 步即可，假定每秒走 N 步，每步之间的间隔为 M 则有

$$N = \frac{4096}{60} \approx 68 \left(\frac{\text{步}}{\text{秒}} \right) \tag{式 3}$$

$$M = \frac{1}{68} \approx 0.014 = 14(\text{毫秒}) \tag{式 4}$$

因为每步之间的间隔过于短暂，因此通过合理配置定时器的计数值，采用定时器中断的方式进行驱动是一种合理的选择。同时为了防止电路中电流过大，采用单相通电的方式进行驱动。同时在系统每一次重新得电后，步进电机也应根据当前时间数值调整至相应的时间位置确保时间的准确性。

3.6 语音交互的设计及原理介绍

集成语音助手，支持自然语言交互，用户可通过语音指令设置闹钟、查询天气等，提升使用便利性。使用 ESP32-S3-N16R8 开发板实现语音识别(ASR)和语音合成(TTS)，通过 I2S 协议传输音频数据，确保高质量的音频处理。结合千帆大模型 API 实现语音指令的处理和响应。

语音交互的粗略流程主要使用语音识别（ASR）和语音合成（TTS）技术，这些技术

目前各大主流厂商都有免费提供。其中 Base64 是一种编码方式，用于将二进制数据转换为纯文本格式。JSON 是一种轻量级的数据交换格式，可在客户端和服务端之间传输数据。

因此其主要原理是先利用麦克风模块将自然语言进行采样储存，随后将该采样数据进行 Base64 编码后封装到 JSON 格式并通过网络链路发送给主流厂商提供的语音识别 API。

随后语音识别 API 将识别语音数据进行转换将其转换为文本，向千帆大模型 API 发送 HTTP POST 请求。解析 API 返回的 JSON 响应，提取模型生成的回答文本^[24]。该模型会根据输入的文本生成回答，同时响应的内容将被用于后续的语音合成。原理如图所示

语音合成环节将千帆大模型返回的回答文本进行 URL 编码，并构造用于语音合成的 HTTP 请求 URL，包含文本内容、音色、语速等参数，最后接收 API 返回的音频数据流。

麦克风和音响模块能够接收和播放音频其原因是因为使用了 I2S 协议，I2S 协议是一种专为数字音频设备设计的串行总线标准，由 Philips Semiconductors（现 NXP Semiconductors）在 1980 年代开发。它主要用于在微控制器、音频处理器、ADC（模数转换器）、DAC（数模转换器）等设备之间传输高质量的数字音频信号。I2S 使用三条主要信号线（时钟线 SCK、数据线 SD 和帧同步线 LRCK）传输音频数据，减少了引脚数量和布线复杂性。

SCK 线提供了同步音频数据传输的时钟信号。确定了数据传输的速度和时序。该时钟也称为 Bit Clock（BCLK）。LRCK 线指示了当前传输的是左声道的音频数据还是右声道的音频数据。它被称为帧同步信号。SD 线用于传输实际的音频数据。数据的位宽可以根据具体应用而变化，通常为 16 位或 32 位。TX 方向为 Serial Data Out（SDOUT），RX 方向为 Serial Data In（SDIN）。

I2S 的配置可以从配置主从模式、采样率，量化位数及声道格式进行配置，这三个参数主要影响存储的音频信号的数据量大小。主从模式的配置中，主设备提供时钟信号（BCLK）和帧同步信号（LRCK），从设备则接收这些信号。主从模式的配置决定了时钟信号的来源和同步方式，对于多设备系统的同步至关重要^[25]。

同时可以使用语音助手帮助完成一些功能，例如将 ESP32-S3-N16R8 同 STM32F103C8T6 进行 UART 连接，双方约定好协议标准后，即可通过语音助手发送指令进而控制 STM32 进行时钟的控制。语音交互示意图如图 26。

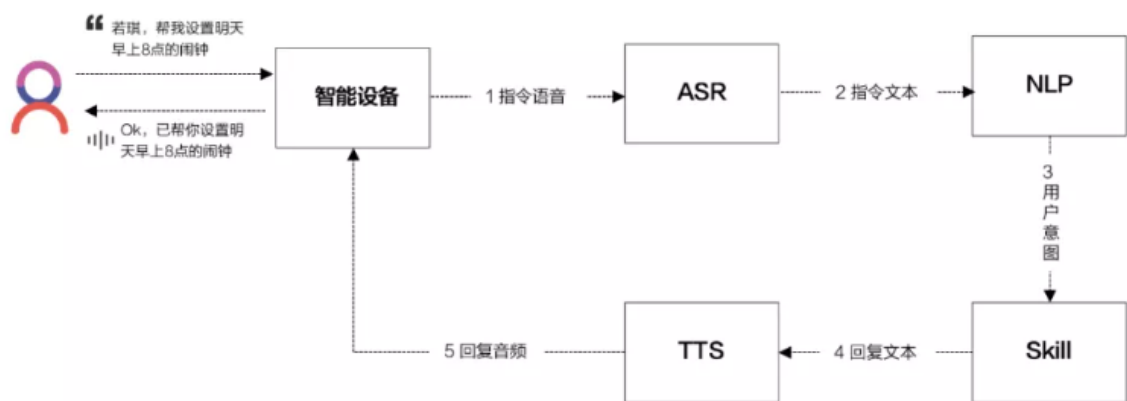


图 26 语音交互流程示意图

4 试验与数据分析

4.1 RTC 时钟精度测试

本次设计中 RTC 时钟使用 LSE 外部低速时钟作为 RTC 时钟源，LSE 精度较高，但是不意味着并不存在误差，其误差由众多因素取决，例如电源电压的稳定性，环境的温度以及 PCB 的布线设计等。测量 LSE 误差的方法可以使用内部定时器中断的方式进行测量。

将定时器的溢出中断频率设置为 LSE 的振荡频率，并将定时器配置为输入捕获模式用于捕获 LSE 的上升沿或下降沿信号。随后在中断回调函数中，捕获 LSE 信号的两个连续上升沿或下降沿的时间戳，并计算它们之间的差值，从而得到实际的 LSE 的周期。通过多次获取周期然后取平均值，最后与理论的 LSE 周期进行比较，计算实际的误差^[26]。

在本次测试中，进行 10 轮测试，每轮测试取 50 个实际的 LSE 周期，假定取平均后实际的 LSE 周期为 T_m ，理论的 LSE 周期为 T_t ，误差周期为 T_e ，则有：

$$T_e = \frac{(T_m - T_t)}{T_t} \times 10^6$$

(式 5)

其中 T_e 的单位为 ppm(百万分之一)，用以表示理论频率和实际频率的差异。由此可以推出周期时间内偏差，假定时间偏差为 S_e ，周期时间为 S_t ，则有：

$$S_e = S_t \times T_e$$

(式 6)

测试条件如下所示：

室温 25℃ ± 2℃

供电电压 3.3V ± 0.05V

每轮采样 50 个周期取平均

表 1 LSE 的误差测试表

序号	实际平均周期 T_m (μ s)	周期误差 T_e (ppm)	时间偏差 S_e (秒/天)
1	30.5192	+23.6	+2.04

2	30.5188	+21.4	-1.85
3	30.5163	-10.9	-0.94
4	30.5205	+31.2	+2.70
5	30.5159	-14.7	-1.27
6	30.5217	+39.8	+3.44
7	30.5144	-26.3	-2.27
8	30.5171	-4.8	-0.41
9	30.5199	+28.5	+2.46
10	30.5223	+43.1	+3.72

对于消费级的实时时钟应用，通常要求精度在 $\pm 20\text{ ppm}$ 到 $\pm 50\text{ ppm}$ 之间,由表 1 可得,该时钟系统满足要求。若需要满足更高的市场需求,可以考虑加入校正系统进行时钟校正,并且校正时间可根据不同日期进行动态调整^[27]。

4.2 按键功能测试

按键作为与系统的重要交互方式之一，需要能够及时的响应，同时需要保证稳定性，因此测试标准为连续按动每个按键 20 次，进行 3 组。观察界面是否卡死并且是否执行相应功能，同时测试对于之间调整这类支持连按的按键是否能会导致 RTC 时钟堵塞。

表 3 按键电路测试

按键序号	响应次数	响应次数	响应次数	无响应次数
1	20	20	20	0
2	20	20	20	0
3	20	20	20	0
4	20	20	20	0
5	20	20	20	0
6	20	20	20	0

由表 1 可得，按键正常工作，并未出现程序异常现象现象。在保险设计下可以采用看门狗复位设计。当程序执行异常时，处理器无法在小于狗叫时间间隔内输出喂狗信号，则看门狗产生复位信号或中断信号，进而使处理器发生复位或中断，最终可使处理器软

件从起始位置开始执行，使软件恢复正常^[28]。图 27 为按键 1 对应的功能界面。

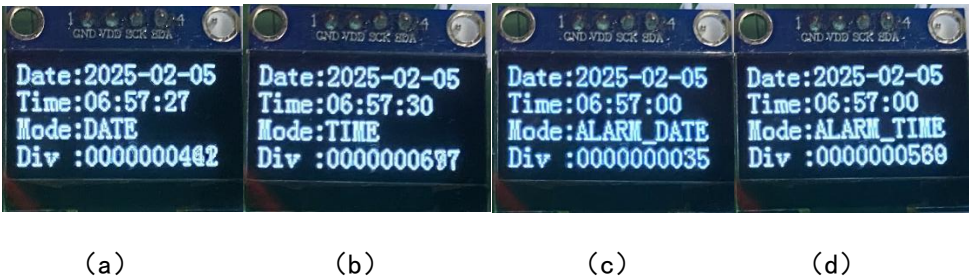


图 27 切换功能按键实物图

4.3 电源尖峰电压测试

由于电感续流以及电路寄生电感的存在,在每次操作开关时均会产生尖峰电压^[29]。示波器抓取的波形如图 28。其中示波器的量程为 5V/格,由图可以得出当前尖峰电压在 18V,该电压会直接影响到后续电路。

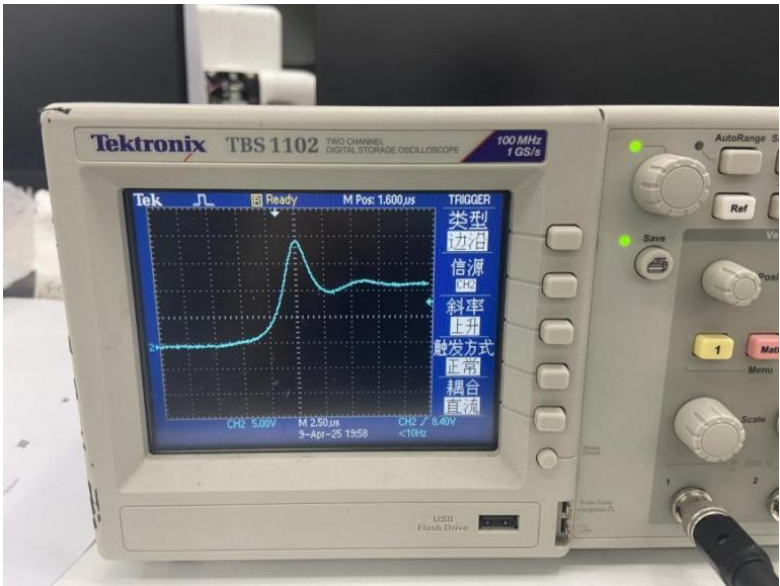


图 28 示波器抓取尖峰电压

而在本设计中，开关后首先接入的即是 12V 转 5V 电路芯片,查阅芯片手册得知 LM2596S-5.0 的输入电压为 4.5V~40V,足以承受该尖峰电压，为了保障芯片的长期工作安全。应多次测试尖峰电压的数值。测试结果如表 4。

表 4 输入电源尖峰电压测试

序号	尖峰电压值(V)
1	14.5
2	13.7
3	17.3
4	16.4
5	15.5
6	13.5
7	15.7
8	15.6
9	17.3
10	18.3

根据可以得知，尖峰电压的峰值在 LM2596S-5.0 承受的电压范围内，并且没有达到临界值，因此可以保障系统长期稳定的工作。如需要抑制尖峰电压，可以采用接入电解电容的方式进行抑制。电路中通过并入适当容性元件，来消除尖峰电压，处于电容两端电压不会突然变化，但是开关电路在换向时会产生瞬间的大电流，电路开启时也很容易出现尖峰电流。而电解电容因为具有较大的容量，能够储存较多的电荷，从而在低频尖峰电流出现时提供稳定的电流输出，抑制尖峰电流对电路的影响^[30]。

5 结论与展望

5.1 结论

本文设计并实现了一种基于单片机的智能实时时钟系统。该系统集成了多种功能，包括精确的时间显示、日期显示、闹钟设置以及通过 WiFi 与外部网络进行语音交互等功能。通过硬件选型、软件设计和系统测试等环节的努力，成功构建了一个稳定可靠、功能完善的智能时钟系统。实验结果表明，该系统能够准确显示时间，并播报天气预报信息，提供便捷的时间调整和闹钟设置功能，既结合了传统时钟，又融入了当下前沿科技。满足作为一个时钟的同时添加额外的互动功能。

在硬件设计方面，本系统采用了单片机作为核心控制器，结合高精度的陶瓷低功耗晶振来确保时间的准确性。并通过 OLED 显示屏清晰地展示时间、日期。此外，系统还集成了语音交互助手，用于获取天气状况及其他可联网搜索的信息。使用蜂鸣器作为闹钟起响铃作用。同时加入蓝牙功放功能用以播放音频，拓宽了智能时钟的使用范围。

在软件设计方面，系统采用了模块化设计方法，将时间显示、日期更新、闹钟设置、网络通信和语音交互等功能分别封装为独立的模块。通过合理的中断处理机制，确保了系统的稳定运行和高效响应。

在系统测试阶段，通过一系列严格的测试，验证了系统的各项功能和性能指标。测试结果表明，系统的时间显示精度高，误差在合理范围内；日期显示准确无误；闹钟功能可靠，能够在设定的时间准时提醒用户。语音交互功能响应迅速，用户可以通过语音指令进行闹钟设置操作。提升了实时时钟的交互功能

5.2 展望

在本文设计中，我们成功实现了一种基于单片机的智能实时时钟系统，该系统不仅具备精确的时间显示和日期显示功能，还集成了闹钟设置、语音交互以及 WiFi 联网等先进功能。通过硬件选型和软件设计的优化，系统展现出较高的稳定性和实用性。未来，我们可以在现有基础上进一步拓展功能，优化性能，使其更具竞争力和实用性。

在功能上仍然可以拓展。考虑集成更多的传感器来丰富系统的功能。例如，通过添加温湿度传感器和空气质量传感器，系统可以实时显示环境数据，提供更全面的健康信息。这不仅可以提升用户体验，还可以使智能时钟系统成为一个多功能的健康监测终端。

在语音交互方面可以增添更多的事件相应，提升对各类外设的交互，例如可以通过语音获取当前环境的空气质量并进行播报，亦或是使用语音对智能时钟的功能进行切换或直接实现该功能。

在硬件设计的优化，步进电机无法做到无延迟跟随系统时钟保持长久的稳定性，这是因为步进电机内部结构决定了不能进行高速运转导致的。可以考虑增加时间补偿机制使得时钟的显示更加精确。

在软件设计方面，我们可以采用更先进的开发工具和编程语言，提高代码的可维护性和可扩展性。使用模块化设计方法，将系统功能分解为多个独立的模块，便于后续的功能扩展和维护^[31]。

在显示效果上的优化，本文的 OLED 在传达信息上的作用功不可没，在未来的完善中可以考虑设计多元的模式显示功能，提升产品的观赏性。同时可以考虑加入子界面模块，使得系统的界面更加简洁。

总之，基于单片机的智能实时时钟系统在未来将不断融合最新的技术成果，展现出更高的性能、更强的功能和更广泛的应用前景。通过持续的创新和优化，我们相信该系统将为人们的生活和工作带来更多便利和创新体验。

参 考 文 献

- [1]沈康,杨俊婷,王颖.基于 AT89C51 单片机的电子时钟[J].数码世界,2020,(11):65-66.
- [2]陈敏.基于 51 单片机的多功能电子钟设计[J].轻工科技,2023,39(01):105-108.
- [3]王波.基于 STM32 单片机的室内空气质量监控系统设计[J].电子测试,2022,36(17):31-33.
- [4]惠州市猫扑科技有限公司.一种智能音箱[P].CN:201721806697.7,2018-09-28.
- [5]王庭良.基于 ARM 芯片的智能家居系统性能优化研究[J].长江信息通信,2024,37(10):104-106.
- [6]ABDULLAH.基于云平台的 STM 32 微控制器智能家居系统设计[D].天津职业技术师范大学,2022.
- [7]叶羽铭.一种压控温补晶体振荡器(VCTCXO)的设计和实现[D].电子科技大学,2020.
- [8]石盼,何东伟,徐博铭.基于 SWD 协议的 Flash 编程分析与实践[J].机电一体化,2020,26(04):20-26.
- [9]吕颖利,赵会娟.基于单片机的步进电机控制系统研究[J].南方农机,2022,53(08):132-134.
- [10]郑振杰,江衍煊,游德智.单片机结合 ULN2003 驱动步进电机[J].电机技术,2010,(06):44-46.
- [11]郭豪,赵树忠.基于单片机的步进电机控制[J].机械工程与自动化,2018,(01):169-170,173.
- [12]PILLANS J. Reducing position errors by vibration optimization of stepper motor drive waveforms[J].IEEE Transactions on Industrial Electronics,2021,68(6):5176-5183.
- [13]罗烜坤,于斌,罗春丽,等.基于 BLE 蓝牙技术的自动气象站调试系统设计[J].陕西气象,2024,(06):68-74.
- [14]贾志成,田祺,周婕妤,等.基于 STM32 的外加式电梯语音交互按键系统[J].吉林大学,2022,40(02):313-318.
- [15]王会彦,朱丽君.物联网设备小型化 PCB 设计的挑战与解决方案[J].中国宽带,2024,20(07):124-126.
- [16]CHENG Y Y, CHIU Y C, HUANG Y C, et al. A DC power connector with voltage spike suppression[C]//2023 11th International Conference on Power Electronics and ECCE Asia (ICPE 2023-ECCE Asia).Jeju Island,Korea,Republic of:IEEE,2023:2525-2529.
- [17]张婉珍,刁立强,刘鑫宇,等.基于 STM32 智能路灯系统的设计与实现[J].物联网技术,2025,15(06):142-144.
- [18]GÓRECKI K, POSOBKIEWICZ K. Influence of a PCB layout design on the efficiency of heat dissipation and mutual thermal couplings between transistors[J].Electronics,2023,12(19):4116.
- [19]赵文龙,钟敏,张鹏.同步实时时钟系统研究[J].南昌航空大学学报(自然科学版),2015,29(02):87-92,97.
- [20]雍文.基于 IIC 总线的温度补偿晶体振荡器数字控制逻辑设计[D].华中科技大学,2010.
- [21]刘正翔.基于 STM32 的 OLED 显示屏接口设计[J].电子技术与软件工程,2017,(19):65.
- [22]李宜书.一种超低功耗 RTC 电路的研究与设计[D].南京邮电大学,2023.
- [23]PRATA S.C primer plus[M]. Upper Saddle River,NJ:Addison-Wesley,2012.

- [24]边玉亮,毛润,李启航,等.基于 ESP32 的 ChatGPT 语音交互系统设计与实现[J].黄冈师范学院学报,2024,44(05):20-25.
- [25]CHEN K. Multimodal interactive intelligent voice interaction system based on artificial intelligence technology[J]. International Core Journal of Engineering,2024,10(8):40-45.
- [26]赵琳莉,王璐,闵莉.基于单片机的时钟校时系统设计[J].科技广场,2009,(01):192-194.
- [27]吴天强,叶敏,朱剑,等.单片机系统实时时钟日差补偿的算法设计[J].科技风,2019,(34):10-11.
- [28]刘宵辰,赵刚,刘卫华.一种嵌入式计算机系统的看门狗设计[J].山西电子技术,2021,(01):32-33.
- [29]刘光清,郑勤.论尖峰电压、电流对开关电源的干扰和抑制措施[J].电子技术与软件工程,2016,(06):116.
- [30]秦博.抑制 DC/DC BOOST 电路尖峰电压及振铃噪声的设计与应用[J].科技创新与应用,2024,14(24):45-49.
- [31]GB/T 7714-2015.ARM debug interface V5 (architecture specification)[S].

附 录

小智 AI 开源项目 Github 仓库: <https://github.com/78/xiaozhi-esp32.git>

主程序源码:

```
#include "stm32f10x.h" // OLED_ShowNum(2, 9, MyRTC_Time[4], 2);
Device header        OLED_ShowNum(2, 12, MyRTC_Time[5], 2);
#include "OLED.h"      MOTOR_Angle_Min(MyRTC_Time[4]/60);
#include "Timer.h"     MOTOR_Angle_Hour(MyRTC_Time[5]/12);
#include "Delay.h"
#include "Rtc_Clock.h"
#include "Key.h"
#include "TIM.h"

volatile int8_t step = 0;          TIM2_Init();

int main(void)
{
    while(1)
    {
        OLED_Init();
        {
            OLED_ShowString(1, 1, "Date:XXXX-XX-XX"
            MyRTC_ReadTime();
        );
            Switch_Change_Mode();
            OLED_ShowString(2, 1, "Time:XX:XX:XX");
            Key_GetNum();
            OLED_ShowString(3, 1, "Mode:");
            Reduce_Num(SetMode , Reduce_Mode);
            OLED_ShowString(4, 1, "Div :");
            Alarm_Clock();
            Alarm_Ring();

            MyRTC_Init();

            Key_Init();
            if((SetMode != ALARM_DATE) &&
            Motor_Init();
            (SetMode != ALARM_TIME))
            MyRTC_ReadTime();
            {
            OLED_ShowNum(1, 6, MyRTC_Time[0], 4);
            OLED_ShowNum(1, 11, MyRTC_Time[1], 2);
            4);
            OLED_ShowNum(1, 14, MyRTC_Time[2], 2);
            OLED_ShowNum(1, 11, MyRTC_Time[1],
            2);
            OLED_ShowNum(2, 6, MyRTC_Time[3], 2);
            2);
```

```

        OLED_ShowNum(1, 14, MyRTC_Time[2],
2);

        OLED_ShowNum(2, 6, MyRTC_Time[3],          OLED_ShowNum(1, 6, MyRTC_Time[0], 4);
2);

        OLED_ShowNum(2, 9, MyRTC_Time[4],          OLED_ShowNum(1, 11, MyRTC_Time[1], 2);
2);

        OLED_ShowNum(2, 12, MyRTC_Time[5],         OLED_ShowNum(1, 14, MyRTC_Time[2], 2);
2);

    }

    //OLED_ShowNum(3, 7, RTC_GetCounter(), 1
0);

    OLED_ShowNum(2, 6, MyRTC_Time[3], 2);

    OLED_ShowNum(2, 9, MyRTC_Time[4], 2);

    OLED_ShowNum(2, 12, MyRTC_Time[5], 2);

    if(SetMode == 0)
        OLED_ShowString(3, 6, "NONE      ");          OLED_ShowNum(4, 6, (32767
-
        if(SetMode == DATE)                          RTC_GetDivider()) / 32767.0 * 999, 10);
        OLED_ShowString(3, 6, "DATE      ");
        if(SetMode == TIME)
        OLED_ShowString(3, 6, "TIME      ");          StepMotor_Run_Min(step);

        if(SetMode == MUSIC)                          StepMotor_Run_Hour(step);
        {
            step = (step + 1) % 4;
            OLED_ShowString(3, 6, "MUSIC
            //Delay_ms(50);
        );
            if(SetMode != MUSIC)
            while(1)//To ensure the step
            break;
        }
        motor 100% Take control the core
        {
            Switch_Change_Mode();
            MyRTC_ReadTime();
            Key_GetNum();//get key          if(SetMode == ALARM_DATE)
            status to break the loop
            {

```

```

OLED_ShowString(3, 6, "ALARM_DATE ");
OLED_ShowNum(2, 12, Alarm_Time[5], 2);
    }

OLED_ShowNum(1, 6, Alarm_Time[0], 4);

OLED_ShowNum(1, 11, Alarm_Time[1], 2);

OLED_ShowNum(4, 6, (32767
    RTC_GetDivider()) / 32767.0 * 999, 10);
    }

OLED_ShowNum(2, 6, Alarm_Time[3], 2);
}

OLED_ShowNum(2, 9, Alarm_Time[4], 2);
void TIM2_IRQHandler(void)
{
    int static Sec_count = 0;
    int static Min_Move = 0;
    if (TIM_GetITStatus(TIM2,
TIM_IT_Update) != RESET)
    {
        TIM_ClearITPendingBit(TIM2,
TIM_IT_Update);
        Sec_count++;
        if (Sec_count == 60)
        {
            StepMotor_Run_Min(Min_Move);
            Min_Move = (Min_Move +
1)%4;
            Sec_count = 0;
        }

OLED_ShowNum(2, 9, Alarm_Time[4], 2);

```

致 谢

本研究的顺利完成，离不开众多师长、同窗、家人以及合作者的无私帮助与支持。值此论文完成之际，谨向所有给予我帮助的人致以诚挚的谢意。

首先，衷心感谢我的导师黄伟锋教授。您严谨的治学态度和渊博的专业知识为我的研究提供了坚实的基础。在嵌入式系统时序同步的关键阶段，您提出的“寄存器状态映射生物节律”的类比，为我打开了全新的思路，使我得以突破技术瓶颈。您的教诲将永远铭刻在我心中。

实验室团队的每一位成员，都是我科研征途上不可或缺的伙伴。在电路板调试和 NTP 校时等关键环节，我们共同攻克了一个又一个技术难题。那些挑灯夜战的日子，共享热风枪温度的瞬间，都将成为我人生中最珍贵的回忆。感谢你们的陪伴与支持。

我的父母，是我永远的避风港与坚强后盾。你们的爱与支持，如同永不偏移的北斗，为我指引着方向。你们的理解与包容，让我能够心无旁骛地专注于学术研究。

此外，还要感谢那些在学术道路上给予我启发与帮助的前辈与同行。你们的论文、报告与讲座，为我打开了通往知识殿堂的大门。感谢学校与学院为我们提供的优越科研条件与平台。

特别感谢“小智 AI”开源项目。这个项目由虾哥开发，是一个基于 ESP32 开发板的开源智能助手项目。它支持多种语言的语音识别和对话功能，为我的研究提供了重要的技术支持和灵感来源。

本研究的完成，是集体智慧的结晶，是多方协作的成果。愿这份成果，能成为团队协作、代际传承与学术探索的见证。